



Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Building Decision Tree Algorithm from Scratch in Python

14 min read · Nov 19, 2023



Bagus Murdyantoro

Follow



Listen



Share



More

Contribute to bagusmur1997/ml_from_scratch development by creating an account on GitHub.

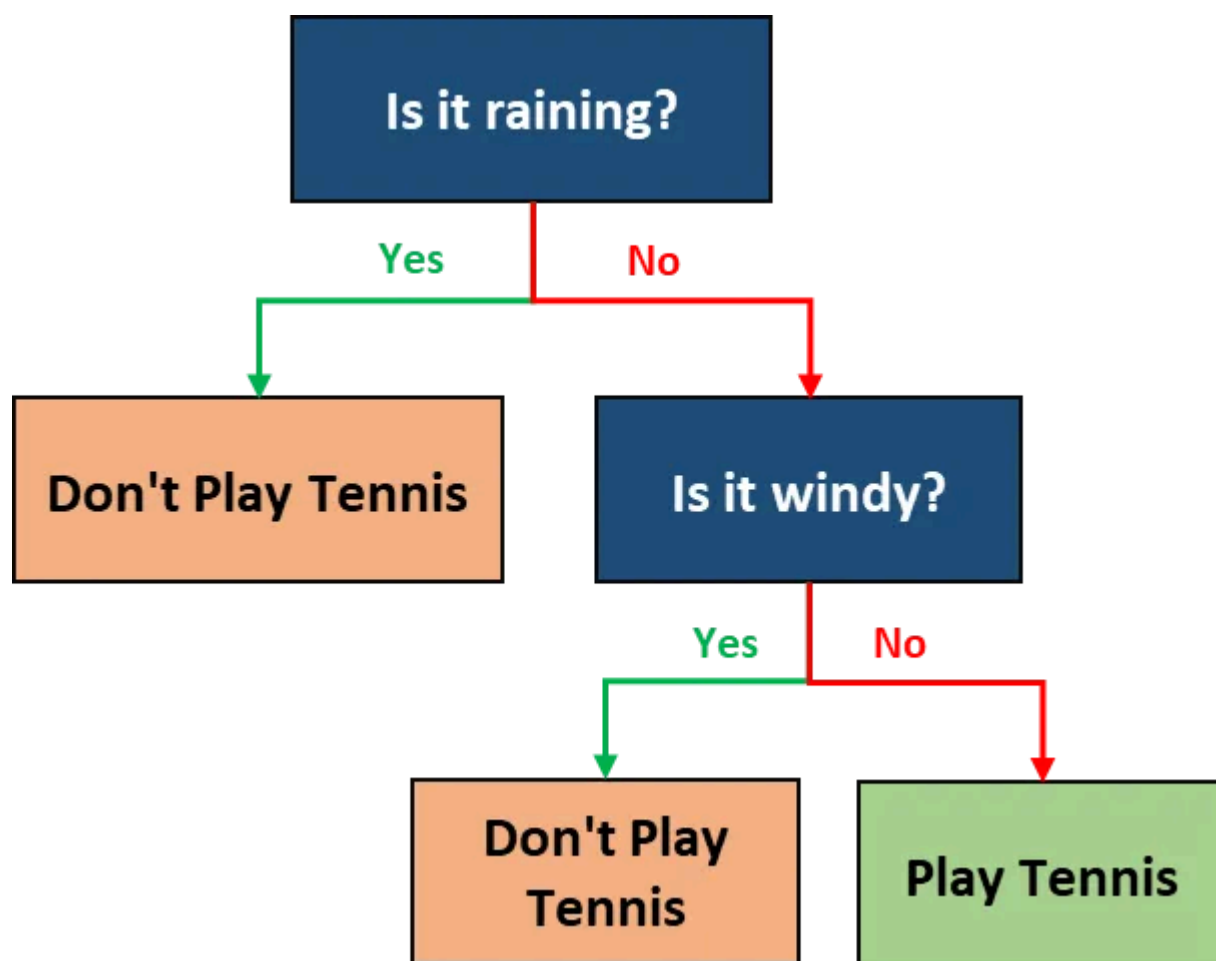
github.com

0 Issues 0 Stars 0 Forks

. . .

Chapter 1 : What is a Decision Tree?

A decision tree is a model that divides the data into subsets based on the value of input features. This division results in a tree structure where each internal node represents a decision based on one of the input features, each branch represents an outcome of that decision, and each leaf node represents a final output or decision (a class label for classification or a continuous value for regression).



Simple Decision Tree Flow Chart

Let's understand the decision tree structure with a simple example, which decides whether to play tennis based on weather conditions. The structure of this decision tree can be visualized as follows:

1. Root Node (Root Node — “Weather Conditions”) : This is the top most node of the tree where The decision-making process begins here, focusing on overall weather conditions.

2. Decision Nodes: These are nodes where the futher decisions are made based on the different attributes.

- **Decision Node 1 — “Is it raining?”:** This node checks if it’s raining to decide the next steps. This decision node come after the root node of decision, Determines the next steps based on rainfall.
- **Decision Node 2 — “Is it windy?”:** Checks wind conditions as the next factor.

3. Leaf Nodes — Outcomes Based on the Decision Tree: These are the final nodes of the tree that provide the decision or outcome that do not splits further.

- **Branch Yes (Windy): Leaf Node: “Don’t Play Tennis” —** Avoid playing tennis in strong winds.
- **Branch No (Not Windy): Leaf Node: “Play Tennis” —** Favorable conditions for tennis, neither rainy nor windy.

4. Branches: These are the links between nodes, representing the outcome of a decision. Branches represent the flow of decisions from each node. Example: From “Is it raining?”, branches lead directly to a decision of not playing tennis if it’s raining. In this decision tree, each decision or node effectively narrows down the conditions, ultimately leading to a clear decision regarding playing tennis based on the weather.

. . .

Chapter 2 : The Decision Tree Algorithm Flow

In this chapter, we’ll explore the step-by-step process of how a decision tree is built, a fundamental concept in machine learning.

- 1. Start at the Root:** Begin with all the training data at the root node and then divided into subsets.
- 2. Select the Best Split:** At each node, find the best split by selecting the feature and the threshold that divide the data in a way that maximizes the homogeneity

or minimize impurity in the resulting subsets.

3. **Create Child Nodes:** Create two child nodes and distribute the data to these nodes based on the split.
4. **Recursion:** Repeat the process for each child node recursively following the same process of selecting the best split until one of the stopping criteria is met (e.g., maximum depth, minimum samples at a node, or no further improvement).
5. **Leaf Node Prediction:** At each leaf node, assign a prediction value. In classification : the most common class label or most frequent label of the samples in that leaf. In regression : the mean/median of the target values.

. . .

Chapter 3 : Impurity and Cost Functions in Decision Tree

Decision trees use various metrics to decide how to split the data. These metrics are crucial in determining the best way to divide the dataset at each step of the tree-building process. To find the best split at each node, we iterate through each feature and its possible thresholds, calculating the impurity reduction. The best split is the one that results in the highest decrease in impurity.

The algorithm selects the best feature and threshold for splitting based on impurity measures. The objective function is to maximize the impurity decrease (for classification) or minimize the error (for regression).

The cost objective/function in a decision tree is to create splits that result in the most homogenous sub-nodes. For classification, this could be minimizing Gini impurity or entropy, and for regression, minimizing the mean squared error.

1. Gini Impurity

Used in classification, **Gini Impurity** measures how often a randomly chosen element would be incorrectly classified. It is calculated using the formula:

$$Gini = 1 - \sum_{i=1}^{classes} (p_i)^2$$

where (p_i) is the proportion of samples that belong to class (i) .

2. Entropy

Entropy is another measure for classification. It indicates the disorder or uncertainty in the data. The formula for entropy is:

$$\text{Entropy} = - \sum_{i=1}^{\text{classes}} p_i \log_2(p_i)$$

where (p_i) is again the proportion of samples that belong to class (i) .

3. Mean Squarred Error (MSE)

Used in regression, **Mean Squared Error (MSE)** indicates the average of the squares of the differences between the actual values and the predicted values. It is given by:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where (y_i) is the actual value and $(\hat{y}_i)$ is the predicted value, and (n) is the number of samples.

These metrics are fundamental in guiding the decision-making process within a decision tree, helping to determine the most informative features and splits.

. . .

Chapter 4 : Calculating Information Gain in Decision Trees

Information Gain measures the effectiveness of a feature in splitting the training data. It quantifies how much information a feature gives us about the class. In decision trees, this concept is used to decide the order of attributes at each node in the tree. Information Gain calculated using this formula :

$$\text{Information Gain} = \text{Current Impurity} - \text{Weighted Impurity after Split}$$

Formulation of Information Gain

where :

- **Current Impurity** is the impurity of the current node before the split. It can be measured using Gini Impurity, Entropy, or Mean Squared Error, depending on the task.
- **Weighted Impurity after Split** is the combined impurity of the two child nodes created by the split, weighted by the number of samples in each child node.

Weighted Impurity after Split

$$= \frac{(\text{Number of Samples in Left Node} \times \text{Impurity Left Node}) + (\text{Number of Samples in Right Node} \times \text{Impurity Right Node})}{\text{Total Number of Samples}}$$

Formulation of Weighted Impurity after Split

Information Gain plays a crucial role in determining where to split the data. Let's break down a typical implementation:

1. **Initial Impurity Calculation:** The algorithm begins by calculating the current impurity of the dataset. This serves as a baseline for measuring improvements.
2. **Iterating Over Features and Thresholds:** The algorithm evaluates each feature and its possible thresholds for splitting the data. It calculates the impurity of the resulting subsets.
3. **Calculating Gain:** For each split, the algorithm calculates the Information Gain. This is done by subtracting the weighted impurity of the subsets from the current impurity.
4. **Selecting the Best Split:** The split that offers the highest Information Gain is chosen. It's the point where the data becomes more organized, and the tree gains the most information.

• • •

Chapter 5: Criteria for Splitting in Decision Trees

When we build decision trees in machine learning, we use certain rules or criteria to decide how to split the data. Think of these like guidelines that help us make the tree just right — not too complex, but detailed enough to make good predictions. Let's look at these rules in a simpler way:

1. `'max_depth'` : The Maximum Depth of the Tree

- This parameter specifies the maximum depth of the tree. The depth of a tree is the length of the longest path from the root to a leaf.
- A deeper tree can model more complex relationships by having more splits and capturing more information about the data.
- However, setting this value too high can lead to overfitting, where the tree becomes too specialized to the training data and performs poorly on unseen data.

2. `'min_samples_split'` : Minimum Number of Samples Required to Split a Node

- This parameter defines the minimum number of samples a node must have before it can be split.
- It acts as a control for pre-pruning. Higher values prevent the model from learning relations which might be highly specific to the particular sample selected for a tree.
- Too high a value can lead to underfitting, making the tree too general and not capturing enough details about the data.

3. `'min_samples_leaf'` : Minimum Number of Samples Required to be at a Leaf Node

- This parameter sets the minimum number of samples that must be present in a leaf node after a split.
- A leaf node with too few samples might indicate that the tree has created a split that is too specific and not representative of the data.
- This parameter is crucial for the tree to generalize well and is another means to control overfitting.

4. `'min_impurity_decrease'` : Threshold for Impurity Decrease

- A node will be split if this split induces a decrease in impurity greater than or equal to this value.
- Impurity measures (such as Gini impurity or entropy in classification, mean squared error in regression) quantify how mixed the target variable is in a set of items.
- This parameter is essential for deciding whether a split is effective enough in reducing uncertainty or disorder in the data, ensuring that each split contributes significantly to discerning the patterns in the data.

. . .

Chapter 6: Pruning Techniques in Decision Trees

Pruning a decision tree is a technique used to reduce the size of the tree and prevent overfitting, which occurs when a model is too complex and captures noise in the training data as if it were significant patterns. This can lead to poor performance on new, unseen data. Pruning aims to improve the tree's generalization ability by simplifying it.

Methods of Pruning:

1. Pre-Pruning (Early Stopping):

- Pre-pruning, also known as early stopping, involves stopping the tree from growing too complex during the training process.
- This can be achieved by setting constraints like maximum depth (`max_depth`), minimum samples per leaf (`min_samples_leaf`), or minimum impurity decrease (`min_impurity_decrease`).
- The growth of the tree is halted once these thresholds are reached, even if further splits could potentially improve the training set accuracy.

2. Post-Pruning (Cost Complexity Pruning):

- Post-pruning is applied after the tree has been fully grown. It involves systematically removing branches that contribute little to the tree's predictive power.

- One common technique is cost complexity pruning, where a complexity parameter (`alpha`) is used to weigh the trade-off between the tree's size and its accuracy.

We will Prune if $(\text{Node Impurity} + \alpha \times \text{Total Leaves}) \leq (\text{Left Impurity} + \text{Right Impurity})$

- The idea is to find and remove branches that increase the tree's predictive error the least for a given increase in tree simplicity.
- The process typically involves:
 1. Growing the tree fully. Calculating the error as branches are incrementally pruned.
 2. Using cross-validation to find the best value of `alpha`.
 3. Pruning the tree using the optimal `alpha` to minimize cross-validated total error.

. . .

Chapter 7: Pseudocode for Decision Tree Algorithm

We're going to explore a decision tree through pseudocode. Pseudocode isn't actual code but more like a blueprint. It's an easy way to understand the steps in building a decision tree without getting into complex code details. Imagine it as a straightforward map guiding us through the process.

Pseudocode for Decision Tree Algorithm:

1. Initialize the Decision Tree

Define a `DecisionTree` class with parameters:

- `max_depth`: Maximum depth of the tree
- `min_samples_split`: Minimum samples required to split a node
- `min_samples_leaf`: Minimum samples required at a leaf node
- `impurity`: Type of impurity measure (e.g., Gini, Entropy)

Initialize root node to `None`

2. Define Node Structure

Define a `Node` class with attributes:

- `feature_index`: Index of the feature used for splitting
- `threshold`: Value to split the feature on
- `left`: Left child node

- right: Right child node
- value: Prediction value at the node
- is_leaf: Boolean indicating if the node is a leaf

3. Calculate Impurity

Define functions to calculate Gini, Entropy, or MSE based on the impurity po

4. Find the Best Split

Define a function to find the best split:

- For each feature in the dataset:
 - For each unique value of the feature as a threshold:
 - Split the dataset into two subsets
 - Calculate the impurity of each subset
 - Calculate the information gain
 - Choose the feature and threshold with the highest information gain

5. Build the Tree

Define a function to build the tree:

- If stopping criteria are met (max depth, min samples), create a leaf node
- Otherwise, find the best split and recursively build left and right subtree

6. Fit the Model

Define a fit function:

- Call the build tree function starting from the root with the entire dataset

7. Predict

Define a predict function:

- For each instance in the input:
 - Traverse the tree based on feature values
 - Return the prediction at the leaf node

8. Traverse the Tree for Prediction

Define a function to traverse the tree for a single instance prediction.

9. Prune the Tree

Define a pruning function:

- For each node in the tree, starting from the leaves:
 - If pruning the node (turning it into a leaf) reduces the complexity of t
 - The pruning decision is based on a complexity parameter (alpha) and valid

End of Decision Tree Algorithm

. . .

Chapter 8: Implementing a Decision Tree in Python

Implementing a decision tree in Python involves understanding several key concepts and translating them into code. Let's break down the process:

1. Understanding the Structure

A decision tree is composed of nodes and branches, where each node represents a decision point based on a feature, and branches represent the outcomes of these decisions.

In Python, each node can be represented as an instance of a `'Node'` class. This class contains attributes like `'feature'` (the feature on which to split), `'threshold'` (the value to split the feature on), `'left'` and `'right'` (child nodes), and `'value'` (the prediction at this node, used in leaf nodes).

```
class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, *,
                  self.feature = feature
                  self.threshold = threshold
                  self.left = left
                  self.right = right
                  self.value = value
                  self.impurity = impurity
                  self.is_leaf = is_leaf
                  self.n_samples = n_samples
```

2. Building the Decision Tree Class

The `'DecisionTree'` class is responsible for the logic of building the tree and making predictions. Key Attributes :

- `'max_depth'` : The maximum depth of the tree.
- `'min_samples_split'` : Minimum number of samples required to consider a split.
- `'min_samples_leaf'` : Minimum number of samples that a leaf node must have.
- `'impurity'` : The measure used to calculate the quality of a split (e.g., Gini, Entropy).

```
def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1,
              min_impurity_decrease=0.0, task='classification', impurity='gini', alph

    self.max_depth = max_depth
    self.min_samples_split = min_samples_split
    self.min_samples_leaf = min_samples_leaf
```

```
self.min_impurity_decrease = min_impurity_decrease
self.task = task
self.impurity = impurity
self.root = None
self.alpha = alpha
```

3. Splitting the Data

One of the crucial steps in building the tree is deciding how to split the data.

A. Calculating Impurity

First, calculate the impurity of the current node. This is done using methods like Gini or Entropy for classification, and Mean Squared Error for regression.

```
def _calculate_gini(self, y):
    _, counts = np.unique(y, return_counts=True)
    probabilities = counts / counts.sum()
    return 1 - np.sum(probabilities**2)

def _calculate_entropy(self, y):
    _, counts = np.unique(y, return_counts=True)
    probabilities = np.clip(counts / counts.sum(), 1e-10, 1)
    return -np.sum(probabilities * np.log2(probabilities))

def _calculate_mse(self, y):
    return np.mean((y - np.mean(y)) ** 2)

def _calculate_impurity(self, y):
    if self.task == 'classification':
        if self.impurity == 'gini':
            return self._calculate_gini(y)
        elif self.impurity == 'entropy':
            return self._calculate_entropy(y)
        else:
            raise ValueError(f"Unknown impurity criterion: {self.impurity}")
    elif self.task == 'regression':
        if self.impurity == 'mse':
            return self._calculate_mse(y)
        else:
            raise ValueError(f"Unknown task: {self.task}")
```

B. Finding the Best Split

Iterate over each feature and its potential thresholds to find the split that results in the highest reduction in impurity.

```

def _best_split(self, X, y):
    n_samples = len(y)
    best_feature, best_threshold, best_gain = None, None, float('-inf')
    current_impurity = self._calculate_impurity(y)
    for feature_idx in range(self.n_features):
        thresholds = np.unique(X[:, feature_idx])
        for threshold in thresholds:
            X_left, y_left, X_right, y_right = self._split_data(X, y, feature_idx, threshold)
            if len(y_left) < self.min_samples_leaf or len(y_right) < self.min_samples_leaf:
                continue
            left_impurity = self._calculate_impurity(y_left)
            right_impurity = self._calculate_impurity(y_right)
            weighted_impurity = (len(y_left) * left_impurity + len(y_right) * right_impurity)
            impurity_gain = current_impurity - weighted_impurity
            if impurity_gain > best_gain and impurity_gain >= self.min_impurity_gain:
                best_feature = feature_idx
                best_threshold = threshold
                best_gain = impurity_gain
    return best_feature, best_threshold

```

4. Building the Tree

Recursively split the data, creating child nodes until the stopping criteria are met (max depth, min samples, etc).

```

def _build_tree(self, X, y, depth=0):
    n_samples = len(y)
    node_impurity = self._calculate_impurity(y)
    node_value = self._calculate_leaf_value(y)
    node = Node(value=node_value, impurity=node_impurity, is_leaf=True, n_samples=n_samples)

    if (self.max_depth is None or depth < self.max_depth) and len(y) >= self.min_samples_split:
        best_feature, best_threshold = self._best_split(X, y)
        if best_feature is not None:
            X_left, y_left, X_right, y_right = self._split_data(X, y, best_feature, best_threshold)
            left_node = self._build_tree(X_left, y_left, depth + 1)
            right_node = self._build_tree(X_right, y_right, depth + 1)

            node.feature = best_feature
            node.threshold = best_threshold
            node.left = left_node
            node.right = right_node
            node.is_leaf = False

```



```
return node
```

5. Making Predictions

To make predictions, traverse the tree from the root to a leaf, following the decision path determined by the splits.

```
def _predict_one(self, x):
    node = self.root
    while node is not None and not node.is_leaf:
        if x[node.feature] <= node.threshold:
            node = node.left
        else:
            node = node.right
    return node.value if node is not None else None

def predict(self, X):
    X = np.array(X)
    return np.array([self._predict_one(sample) for sample in X])
```

Chapter 8: Conclusion and Future Works

In this article, We have traveled through the process of developing decision tree algorithm from scratch in Python in this article. Starting from the ground level, we got to know that decision trees are and how they work. Throughout different chapters this work focused on important issues like the decision making process, purity and cost function, information gain, and the node splitting criteria. We also considered pruning methods to improve the model.

They were made real when their practical implementation in Python was used to create and employ a decision tree for both classification and regression purposes. The practice reinforces understanding, and it serves as an important application of this in everyday life.

A good insight into the decision tree algorithms serves as a great foundation for exploring more complex machine learning techniques. Future work could focus on

several key areas:

1. Random Forests:

- Investigating how combining multiple decision trees into a Random Forest improves model performance.
- Exploring feature importance and how Random Forests can be used for feature selection.
- Optimizing Random Forests for various types of data and problem scenarios.

2. Boosting Techniques:

- Examining different boosting algorithms like AdaBoost, Gradient Boosting, and XGBoost.
- Understanding how boosting improves weak learners and its impact on model accuracy.
- Implementing boosting algorithms in Python and comparing their performance with basic decision trees.

3. Hybrid Models:

- Combining decision trees with other machine learning techniques to create hybrid models.
- Studying the effectiveness of hybrid models in specific applications like time series analysis, image processing, or natural language processing.

4. Large Scale Data Handling:

- Optimizing decision tree algorithms for big data scenarios using techniques like parallel processing or distributed computing.
- Integrating decision tree models with big data platforms like Hadoop or Spark.

5. Real-World Application and Case Studies:

- Applying advanced decision tree models to real-world problems in various domains such as finance, healthcare, or environmental studies.

- Documenting detailed case studies to showcase the practical effectiveness of these models.



Follow

Written by Bagus Murdyantoro

8 followers · 13 following

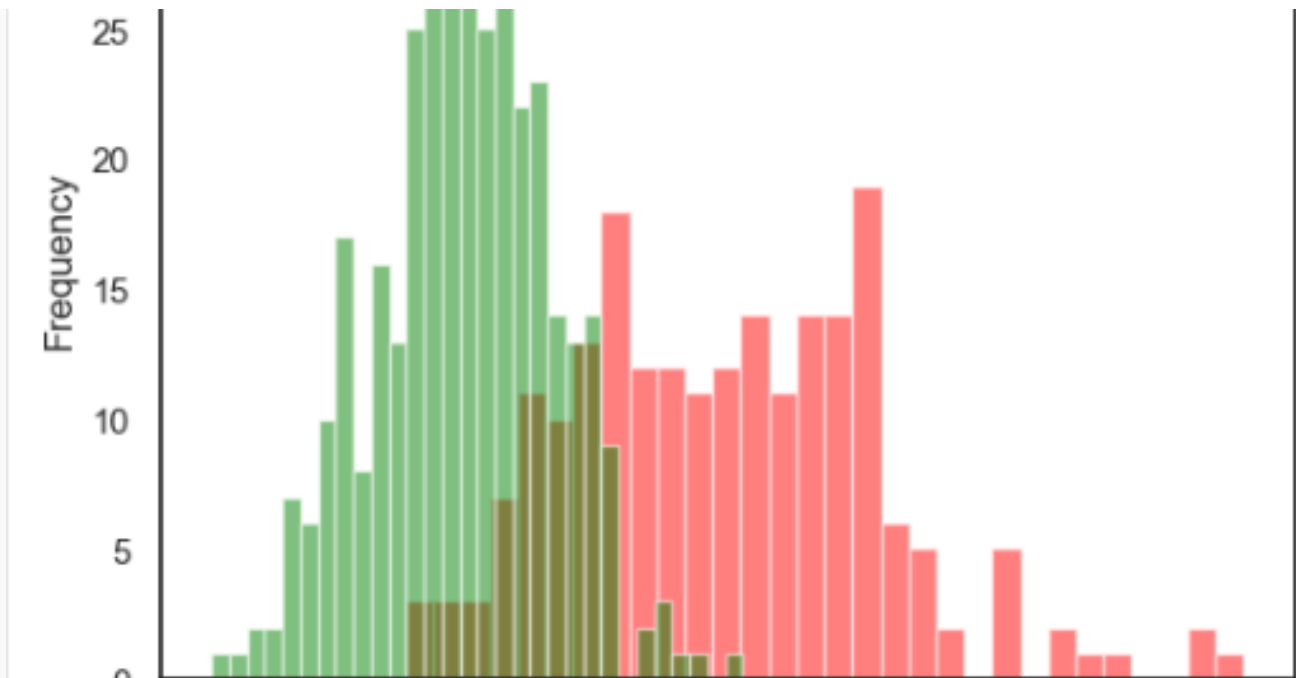
No responses yet



Ashfak

What are your thoughts?

More from Bagus Murdyantoro



 Bagus Murdyantoro

Statistical Analysis of Breast Cancer Wisconsin (Diagnostic)

Introduction

Jul 9, 2023  28



 Bagus Murdyantoro

Exploratory Data Analysis Reveals Insights in OList-Ecommerce Dataset

In this article, we embark on an exciting journey through the OList-Ecommerce dataset, using Exploratory Data Analysis (EDA) to uncover...

Jul 14, 2023



Bagus Murdyantoro

Data Analysis—Shipments Dashboard

In the logistics industry, timely delivery of shipments is a key factor for a business's success. With the demand for faster deliveries...

Apr 15, 2023



5



Bagus Murdyantoro

Building Transformer from Scratch in PyTorch

Introduction

Mar 9, 2024 🖱 3



See all from Bagus Murdyantoro

Recommended from Medium



Olivia Godwin

Diabetes Prediction with Machine Learning by Model Mavericks

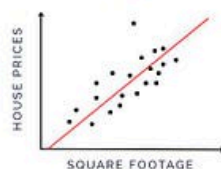
U

Jun 13

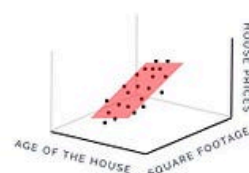


TOP 10 MACHINE LEARNING ALGORITHMS EXPLAINED

Linear Regression



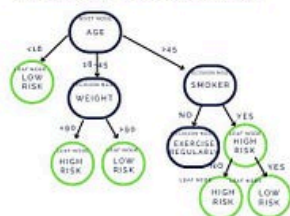
Multiple Linear Regression



Logistic Regression



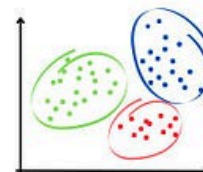
Decision Tree Classifier



K-Nearest Neighbour



K-Means

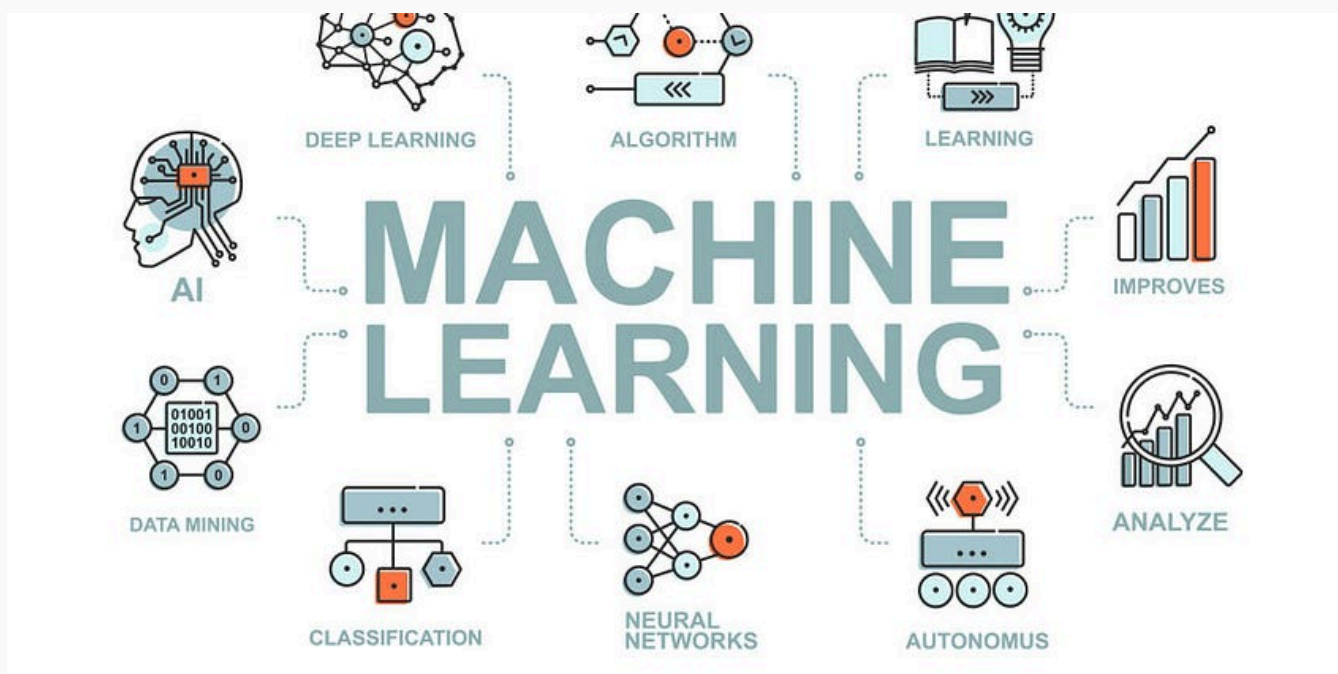


In Learning Data by Rita Angelou

10 ML Algorithms Every Data Scientist Should Know—Part 1

I understand well that machine learning might sound intimidating. But once you break down the common algorithms, you'll see they're not.

✦ Jun 10 🖱 11



October

Machine Learning basics in 10 minutes

Machine learning is how machines learn to do stuff. Stuff could be predicting the next apocalypse or your grocery bill based on some data...

★ Jun 3 🖱 54

**AI** In Artificial Intelligence in Plain English by Kushagra Pandya

🧠 Predicting Diabetes Using Machine Learning: A Complete Workflow

In this article, we walk through a full machine learning pipeline for predicting diabetes using patient data. We'll cover everything from...

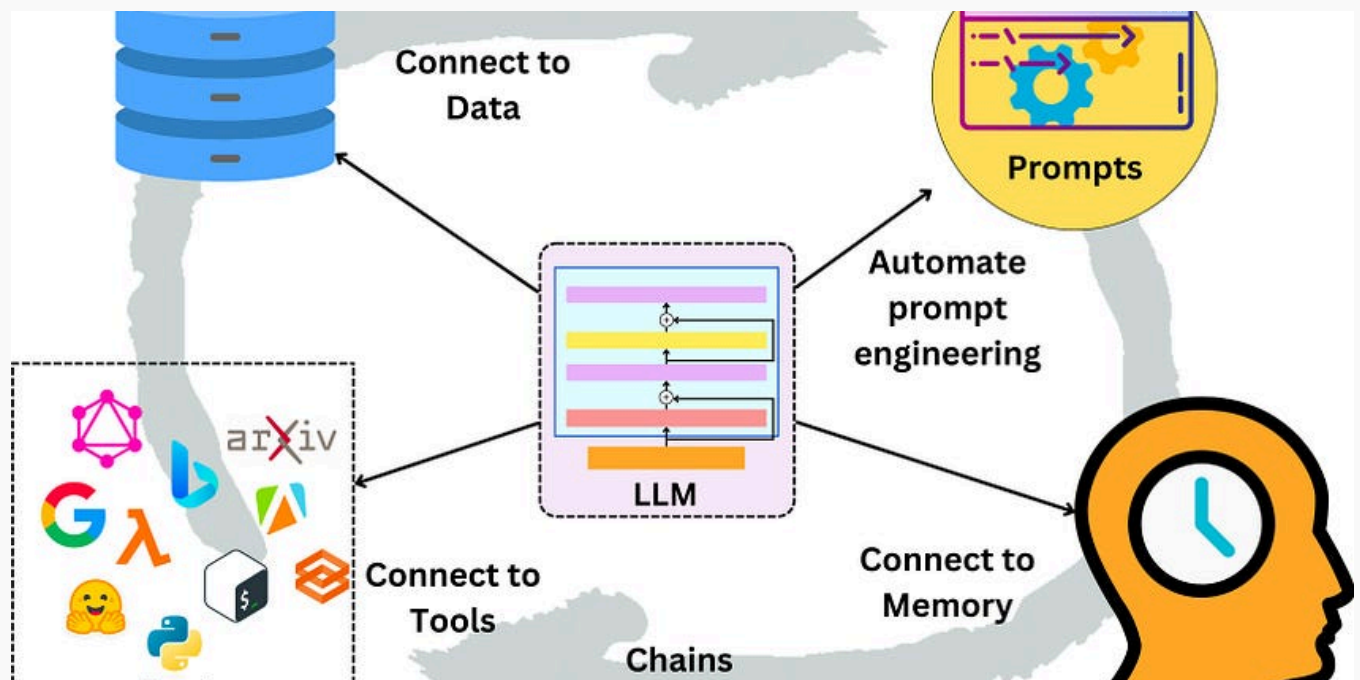
★ Jun 21 🖱 9

 Damini Vadrevu

What are Decision Trees, Random Forest and Gradient Boosting Models?

An all about Ensemble.

★ Mar 26 🖱 6



Piyush Agnihotri

Generative AI with LangChain: Chains & Output Parsers (Part 3)

Deep dive into building real-world generative AI applications using LangChain—from prompts to agents and beyond.

★ Jun 26 🖱 323 💬 3



See more recommendations